



OBJECTIFS DE LA SEANCE

À la fin de cette séance, vous serez capable de :

- Comprendre et résoudre le problème du "prop drilling"
- Maîtriser Context API de React
- Créer un système d'authentification global
- Gérer un panier de location partagé
- Optimiser les performances avec Context

PREREQUIS

- TP 1 à 5 terminées
- Compréhension de useState et useEffect
- React Router fonctionnel

PARTIE 1 : LE PROBLEME DU PROP DRILLING (20 MIN)

1.1 QU'EST-CE QUE LE PROP DRILLING ?

DEFINITION

Passer des props sur plusieurs niveaux de composants pour atteindre un composant profond.

EXEMPLE DU PROBLEME

//  Prop Drilling - Mauvaise pratique

```
function App() {
  const [user, setUser] = useState(null);
  return <Layout user={user} setUser={setUser} />;
}

function Layout({ user, setUser }) {
  return (
    <div><Header user={user} setUser={setUser} />
    <Main user={user} /></div>;
  )
}

function Header({ user, setUser }) {
  return (
    <nav><Logo />
    <Menu user={user} setUser={setUser} />
    </nav> );
}

function Menu({ user, setUser }) {
  return <UserMenu user={user} setUser={setUser} />;
}

function UserMenu({ user, setUser }) {
  // Enfin ! On utilise user ici
  return <div>{user?.name}</div>;
}
```

Problèmes :

- Code verbeux et répétitif
- Composants intermédiaires reçoivent des props qu'ils n'utilisent pas
- Difficile à maintenir
- Difficile de refactoriser

//  Context API - Bonne pratique

```
const UserContext = createContext();

function App() {
  const [user, setUser] = useState(null);

  return (
    <UserContext.Provider value={{ user, setUser }}>
      <Layout />
    </UserContext.Provider>
  );
}

function Layout() {
  return (
    <div>
      <Header />
      <Main />
    </div>
  );
}

function UserMenu() {
  const { user } = useContext(UserContext);
  // Accès direct sans prop drilling !
  return <div>{user?.name}</div>;
}
```



PARTIE 2 : CREER UN CONTEXT (30 MIN)

2.1 ANATOMIE D'UN CONTEXT

3 étapes pour créer un Context :

// 1. Créer le Context

```
const MonContext = createContext();
```

// 2. Créer le Provider

```
function MonProvider({ children }) {  
  const [state, setState] = useState(valeurInitiale);
```

```
  const value = {  
    state,  
    setState,  
    // autres fonctions utiles  
  };  
};
```

```
  return (  
    <MonContext.Provider value={value}>  
      {children}  
    </MonContext.Provider>  
  );  
}
```

// 3. Créer un hook personnalisé (optionnel mais recommandé)

```
function useMonContext() {  
  const context = useContext(MonContext);  
  
  if (!context) {  
    throw new Error('useMonContext must be used within MonProvider');  
  }  
  
  return context;  
}
```



2.2 EXEMPLE SIMPLE : THEME CONTEXT

```
import { createContext, useState, useContext } from "react";

const ThemeContext = createContext();

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("dark");

  const toggleTheme = () => {
    setTheme((prev) => (prev === "dark" ? "light" : "dark"));
  };

  const value = {
    theme,
    toggleTheme,
  };

  return (
    <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>
  );
}

export function useTheme() {
  const context = useContext(ThemeContext);

  if (!context) {
    throw new Error("useTheme must be used within ThemeProvider");
  }

  return context;
}
```



UTILISATION

DANS APP.JSX:

```
// Dans App.jsx
import { ThemeProvider } from '../context/ThemeContext';

function App() {
  return (
    <ThemeProvider>
      <BrowserRouter>
        <Routes>
          { /* ... */ }
        </Routes>
      </BrowserRouter>
    </ThemeProvider>
  );
}
```

DANS UN COMPOSANT

```
import { useTheme } from '../context/ThemeContext';

function Header() {
  const { theme, toggleTheme } = useTheme();

  return (
    <div className={theme === 'dark' ? 'bg-black' : 'bg-white'}>
      <button onClick={toggleTheme}> Changer le thème</button>
    </div>
  );
}
```



🔒 PARTIE 3 : AUTHCONTEXT - AUTHENTIFICATION GLOBALE (40 MIN)

- Créez le provide Context/AuthProvider.jsx

```
import { createContext, useContext, useState, useEffect } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  // TODO : Charger l'utilisateur depuis localStorage au démarrage

  // Fonction de connexion
  const login = async (email, password) => {
    try {
      // TODO: Sera remplacé plus tard par la vraie API (séance 8 ou 9 , ça dépend ;-))
      // Simulation
      await new Promise(resolve => setTimeout(resolve, 1000));

      const mockUser = {
        id: Date.now(),
        email: email,
        name: email.split('@')[0],
        avatar: `https://ui-avatars.com/api/?name=${email}&background=e50914&color=fff`
      };

      // TODO : Enregistrer les données de l'utilisateur dans le localStorage
      return { success: true };
    } catch (error) {
      return { success: false, error: error.message };
    }
  };

  // Fonction d'inscription
  const register = async (name, email, password) => { // TODO : inspirez-vous de plus haut };

  // Fonction de déconnexion
  const logout = () => { // TODO : Supprimez l'utilisateur enregistré en mémoire: };

  // Vérifier si l'utilisateur est connecté
  const isAuthenticated = () => { //XXXX };

  // Mettre à jour le profil
  const updateProfile = (updates) => {
    const updatedUser = { ...user, ...updates }; //ça ne vous rappelle rien ?
    // TODO : Mettre à jour et stocker l'utilisateur
  };
}
```



```
//On met à disposition les éléments pour être utilisés dans les composants
const value = {user, loading, login, register, logout, isAuthenticated, updateProfile };

return (
  <AuthContext.Provider value={value}> {!loading && children} </AuthContext.Provider>
);
}

// Hook personnalisé
export function useAuth() {
  const context = useContext(AuthContext);

  if (!context) throw new Error('useAuth must be used within AuthProvider');

  return context;
}
```

3.2 INTEGRER AUTHCONTEXT DANS L'APPLICATION

- Modifiez App.jsx

3.3 UTILISER AUTHCONTEXT DANS LES COMPOSANTS

- Modifiez le composant Login

```
/** ... **/
const { login } = useAuth();

const handleSubmit = async (e) => {
  e.preventDefault();

  const newErrors = validateForm();
  if (Object.keys(newErrors).length > 0) {
    setErrors(newErrors);
    return;
  }

  setLoading(true);

  // Simulation de connexion
  const result = await login(formData.email, formData.password);

  if (result.success) {
    navigate("/");
  } else {
    setApiError(result.error || "Erreur de connexion");
    setLoading(false);
  }
};
```



- Modifiez le composant NavBar

```
/** ... */
function NavBar() {
  const { user, logout, isAuthenticated } = useAuth();

  /** */
  /* ne pas oublier de coder la fonction de logout*/
  return (
    /** */

    { /* User Section */ }
    <div className="flex items-center space-x-4">
      { /* <SearchBar movies={movies} onSearch={onSearch} /> */ }

      <button className="hover:text-gray-300 transition-colors">
        <svg
          className="w-6 h-6"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            strokeWidth={2}
            d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
          />
        </svg>
      </button>
      {isAuthenticated() ? (
        <div className="relative">
          <button
            onClick={() => setShowUserMenu(!showUserMenu)}
            className="flex items-center space-x-2"
          >
            <img
              src={user.avatar}
              alt={user.name}
              className="w-8 h-8 rounded cursor-pointer hover:ring-2 hover:ring-primary transition"
            />
            <span className="hidden md:block text-sm">{user.name}</span>
          </button>

          {showUserMenu && (
            <div className="absolute right-0 mt-2 w-48 bg-black/95
              backdrop-blur-lg border border-gray-800 rounded-lg shadow-xl py-2">
              <NavLink
                to="/profile"

```



```

        className="block px-4 py-2 hover:bg-gray-800 transition"
        onClick={() => setShowUserMenu(false)}>
        Mon profil
      </NavLink>
      <NavLink
        to="/my-rentals"
        className="block px-4 py-2 hover:bg-gray-800 transition"
        onClick={() => setShowUserMenu(false)}>
        Mes locations
      </NavLink>
      <hr className="border-gray-800 my-2" />
      <button
        onClick={handleLogout}
        className="w-full text-left px-4 py-2 hover:bg-gray-800 transition text-red-400">
        Déconnexion
      </button>
    </div>
  )}
</div>
) : (
  <Link to="/login">
    <button className="px-4 py-2 bg-primary hover:bg-primary-dark rounded transition">
      Connexion
    </button>
  </Link>
)
  )}

```

/** **/

- Modifier ProtectedRoute.jsx :





PARTIE 4 : CARTCONTEXT - GESTION DU PANIER (40 MIN)

4.1 CREER LE CARTCONTEXT

- Créez le fichier context/CartContext.jsx :

```
import { createContext, useContext, useState, useEffect } from 'react';

const CartContext = createContext();

export function CartProvider({ children }) {

  // Todo : Chargez et initialisez le panier et les locations

  // Todo : Sauvegardez le panier et les locations à chaque modif

  // Ajouter au panier
  const addToCart = (movie) => { //Todo :XXX };

  // Retirer du panier
  const removeFromCart = (movieId) => { //Todo :XXX };

  // Vider le panier
  const clearCart = () => { //Todo :XXX };

  // Calculer le total
  const getCartTotal = () => { //Todo :XX }

  // Nombre d'items
  const getCartCount = () => { //Todo :XXX };

  // Louer un film
  const rentMovie = (movie) => {
    const rentalDate = new Date();
    const expiryDate = new Date();
    expiryDate.setDate(expiryDate.getDate() + 7); // 7 jours

    const rental = {
      id: Date.now(),      movieId: movie.id,      title: movie.title,
      poster: movie.poster, price: movie.price,
      rentalDate: rentalDate.toISOString(),
      expiryDate: expiryDate.toISOString()
    };

    //Todo : Mettre à jour la liste des films loués
    //Supprimer le film du panier

    return { success: true, rental };
  };
};
```



```
// Louer tous les films du panier
const rentAllInCart = () => {
  //Todo :XXX
  //Todo :vider le panier

  return { success: true, count: newRentals.length };
};

// Vérifier si un film est loué
const isRented = (movieId) => { //Todo :XXX };

// Obtenir la location d'un film
const getRentalByMovieId = (movieId) => { //Todo :XXX };

// Vérifier si un film est dans le panier
const isInCart = (movieId) => { //Todo :XXX };

const value = {
  cart,
  rentals,
  addToCart,
  removeFromCart,
  clearCart,
  getCartTotal,
  getCartCount,
  rentMovie,
  rentAllInCart,
  isRented,
  getRentalByMovieId,
  isInCart
};

return (
  <CartContext.Provider value={value}>
    {children}
  </CartContext.Provider>
);
}

export function useCart() {
  const context = useContext(CartContext);

  if (!context) {
    throw new Error('useCart must be used within CartProvider');
  }

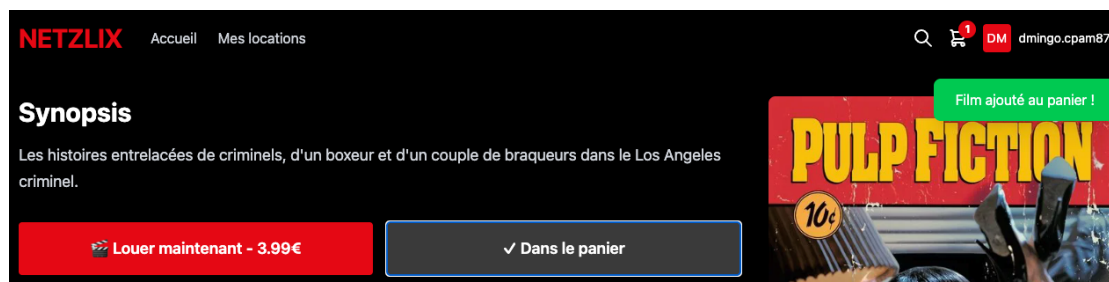
  return context;
}
```

4.2 INTEGRER CARTCONTEXT

- Ajoutez à App le nouveau provider
- Créez ou modifiez le composant common/CartButton
- Intégrez le CartButton à la Navbar

4.4 MODIFIEZ LA PAGE MOVIEDETAIL

- Modifiez la page pour permettre d'ajouter un film dans le panier ou le louer directement





4.4 CREEZ LA PAGE CART

pages/Cart.jsx :

```
function Cart() {

  //Todo : .....

  const handleRentAll = () => {
    //Todo : Si l'utilisateur n'est pas connecté => retour au login

    const result = rentAllInCart();
    if (result.success) {
      alert(`${result.count} film(s) loué(s) avec succès !`);
      //Todo : affichez la page des locations
    }
  };

  if (cart.length === 0) {
    return (
      <div className="min-h-screen bg-black text-white">
        <Navbar />

        <div className="container mx-auto px-4 py-24">
          <h1 className="text-4xl font-bold mb-8">Mon Panier</h1>

          <div className="text-center py-20">
            <svg className="w-24 h-24 mx-auto mb-6 text-gray-600" fill="none" stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M3 3h2l.4 2M7 13h10l4-8H5.4M7 13L5.4 5M7 13l-2.293
              2.293c-.63.63-.184 1.707.707 1.707H17m0 0a2 2 0 100 4 2 2 0 000-4zm-8 2a2 2 0 11-4 0 2 2 0 014 0z" />
            </svg>
            <p className="text-2xl text-gray-400 mb-6">Votre panier est vide</p>
            <Button>Découvrir les films</Button>
          </div>
        </div>

        <Footer />
      </div>
    );
  }

  return (
    <div className="min-h-screen bg-black text-white">
      <Navbar />

      <div className="container mx-auto px-4 py-24">
        <h1 className="text-4xl font-bold mb-8">Mon Panier</h1>

        <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
```



```

{/* Liste des films */}
<div className="lg:col-span-2 space-y-4">
  //Todo : pour chaque film du panier
  <div key={movie.id} className="flex items-center bg-gray-900 rounded-lg p-4 border border-gray-800">
    <img src={movie.poster} alt={movie.title} className="w-20 h-28 object-cover rounded"
    />

    <div className="flex-1 ml-4">
      <h3 className="text-xl font-bold mb-1">{movie.title}</h3>
      <p className="text-gray-400 text-sm mb-2">
        {movie.year} • {movie.genre} • {movie.duration} min
      </p>
      <p className="text-primary font-bold text-lg">
        {movie.price.toFixed(2)}€
      </p>
    </div>

    <button className="text-red-400 hover:text-red-300 transition ml-4" >
      <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24">
        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M19 7l-.867 12.142A2
        2 0 0 1 16.138 21H7.862a2 2 0 0 1 -1.995 -1.858L5 7m5 4v6m4-6v6m1-10V4a1 1 0 0 1 -1h-4a1 1 0 0 1 -1v3M4
        7h16" />
      </svg>
    </button>
  </div>
  //Todo : fin film
</div>

{/* Résumé */}
<div>
  <div className="bg-gray-900 rounded-lg p-6 border border-gray-800 sticky top-24">
    <h2 className="text-2xl font-bold mb-6">Résumé</h2>

    <div className="space-y-3 mb-6">
      <div className="flex justify-between text-gray-400">
        <span>Nombre de films:</span>
        <span>{cart.length}</span>
      </div>
      <div className="flex justify-between text-gray-400">
        <span>Durée de location:</span>
        <span>7 jours</span>
      </div>
    </div>
    <hr className="border-gray-800" />
    <div className="flex justify-between text-2xl font-bold">
      <span>Total:</span>
      <span className="text-primary">{getCartTotal().toFixed(2)}€</span>
    </div>
  </div>

  <Button onClick={handleRentAll} className="w-full mb-3">
    Louer tout ({cart.length} film{cart.length > 1 ? 's' : ''}) </Button>

```



```

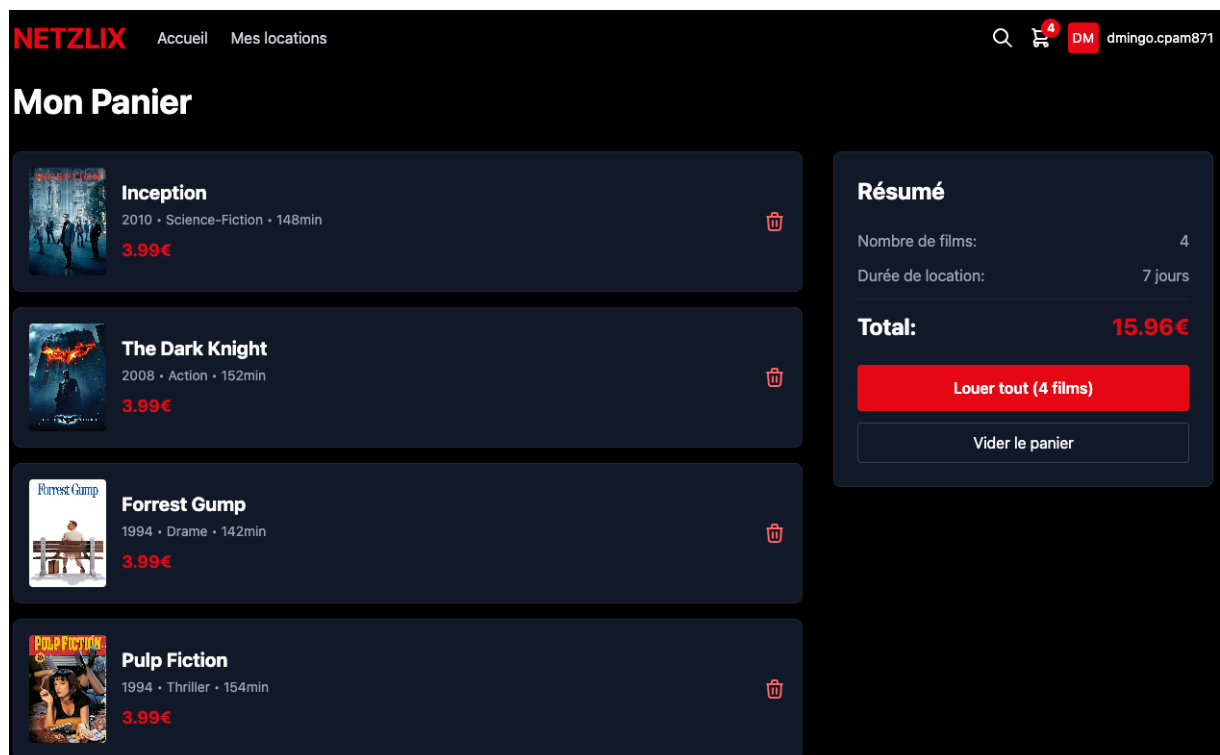
    <button
      onClick={clearCart}
      className="w-full px-4 py-2 border border-gray-700 rounded hover:bg-gray-800 transition"
    >
      Vider le panier
    </button>
  </div>
</div>
</div>
</div>

<Footer />
</div>
);
}

export default Cart;

```

- Modifiez le CartButton pour afficher la page Cart





PARTIE 5 : ON CONTINUE

- Modifiez la page register pour utiliser AuthContext
- Modifiez la page myRentals pour utiliser le CartContext

```
function MyRentals() {

  //Todo : Filtrer les locations actives (non expirées)
  const activeRentals = [] ;

  // Todo : Filtrer les locations expirées
  const expiredRentals = [] ;

  return (
    <div className="min-h-screen bg-black text-white">
      <Navbar />

      <div className="container mx-auto px-4 py-24">
        <h1 className="text-4xl font-bold mb-8">Mes locations</h1>

        { /* Locations actives */ }
        { activeRentals.length > 0 && (
          <div className="mb-12">
            <h2 className="text-2xl font-bold mb-4 text-green-400">
              Actives ( {activeRentals.length} )
            </h2>
            <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-4 xl:grid-cols-5 gap-4">
              {activeRentals.map(rental => (
                <div key={rental.id} className="group relative">
                  <Link to={` /movie/${rental.movieId} `}>
                    <img
                      src={rental.poster}
                      alt={rental.title}
                      className="w-full rounded-lg group-hover:scale-105 transition-transform
duration-300"
                    />
                    <div className="mt-2">
                      <h3 className="font-semibold truncate">{rental.title}</h3>
                      <p className="text-sm text-green-400">
                        Expire dans {Math.ceil((new Date(rental.expiryDate) - new Date()) / (1000 *
60 * 60 * 24))} jour(s)
                      </p>
                    </div>
                  </div>
                )
              )}
            </div>
          </div>
        )
        }
      </div>
    </div>
  )
}
```




```

        </div>
      </Link>
    </div>
  )))
</div>
</div>
))

{/* Locations expirées */}
{expiredRentals.length > 0 && (
  <div>
    <h2 className="text-2xl font-bold mb-4 text-gray-400">
      Expirées ({expiredRentals.length})
    </h2>
    <div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-4 xl:grid-cols-5 gap-4">
      {expiredRentals.map(rental => (
        <div key={rental.id} className="opacity-50">
          <img
            src={rental.poster}
            alt={rental.title}
            className="w-full rounded-lg grayscale"
          />
          <div className="mt-2">
            <h3 className="font-semibold truncate">{rental.title}</h3>
            <p className="text-sm text-red-400">Expiré</p>
          </div>
        </div>
      )))
    </div>
  </div>
)}

{/* Aucune location */}
{rentals.length === 0 && (
  <div className="text-center py-20">
    <svg className="w-24 h-24 mx-auto mb-6 text-gray-600" fill="none"
      stroke="currentColor" viewBox="0 0 24 24">
      <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M7
4v16M17 4v16M3 8h4m10 0h4M3 12h18M3 16h4m10 0h4M4 20h16a1 1 0 001-1V5a1 1 0
00-1-1H4a1 1 0 00-1 1v14a1 1 0 001 1z" />
    </svg>
    <p className="text-2xl text-gray-400 mb-6">Aucune location pour le moment</p>
  </div>
)}

```

```

    <Link to="/">
      <button className="px-6 py-3 bg-primary hover:bg-primary-dark rounded-lg font-
semibold transition">
        Découvrir des films
      </button>
    </Link>
  </div>
  })
</div>

<Footer />
</div>
);
}

export default MyRentals;

```

Mes locations

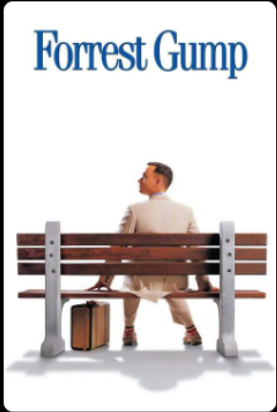
Actives (4)



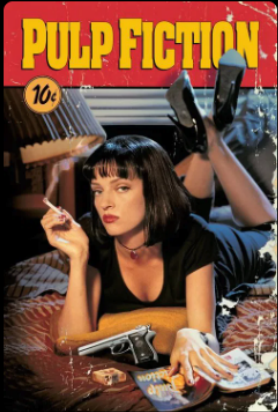
Inception
Expire dans 7 jour(s)



The Dark Knight
Expire dans 7 jour(s)



Forrest Gump
Expire dans 7 jour(s)



Pulp Fiction
Expire dans 7 jour(s)



🎯 CHALLENGE BONUS : NOTIFICATIONCONTEXT

OBJECTIF

Créer un système de notifications toast réutilisable.

FONCTIONNALITES

- Afficher des notifications dans toute l'application
- Auto-dismiss après 3 secondes
- Support de différents types (success, error, info, warning)
- File d'attente de notifications

STARTERCODE

```
import { createContext, useState, useContext } from 'react';

const NotificationContext = createContext();

export function NotificationProvider({ children }) {
  const [notifications, setNotifications] = useState([]);

  const addNotification = (message, type = 'info') => {
    const id = Date.now();
    const notification = { id, message, type };

    setNotifications(prev => [...prev, notification]);

    // Auto-dismiss après 3 secondes
    setTimeout(() => {
      removeNotification(id);
    }, 3000);
  };

  const removeNotification = (id) => {
    setNotifications(prev => prev.filter(n => n.id !== id));
  };

  // Fonctions helper
  const success = (message) => addNotification(message, 'success');
  const error = (message) => addNotification(message, 'error');
  const info = (message) => addNotification(message, 'info');
  const warning = (message) => addNotification(message, 'warning');

  const value = { notifications, addNotification, removeNotification, success, error, info, warning };
}
```



```

return (
  <NotificationContext.Provider value={value}>
    {children}
    <ToastContainer notifications={notifications} onClose={removeNotification} />
  </NotificationContext.Provider>
);
}

function ToastContainer({ notifications, onClose }) {
  return (
    <div className="fixed top-20 right-4 z-50 space-y-2">
      {notifications.map(notification => (
        <Toast key={notification.id} notification={notification} onClose={() => onClose(notification.id)} />
      ))}
    </div>
  );
}

function Toast({ notification, onClose }) {
  const colors = { success: 'bg-green-500', error: 'bg-red-500', info: 'bg-blue-500', warning: 'bg-yellow-500' };

  return (
    <div className={` ${colors[notification.type]} text-white px-6 py-3 rounded-lg shadow-xl flex items-center space-x-3 animate-slide-in`} >
      <span>{notification.message}</span>
      <button onClick={onClose} className="hover:text-gray-200">× </button>
    </div>
  );
}

export function useNotification() {
  const context = useContext(NotificationContext);

  if (!context) throw new Error('useNotification must be used within NotificationProvider');
  return context;
}

```

CHECKLIST DE FIN DE SEANCE

- AuthContext créé et fonctionnel
- CartContext créé et fonctionnel
- Connexion/Déconnexion fonctionne
- Panier fonctionnel
- Page Cart créée
- MyRentals utilise CartContext
- Au moins 2 exercices terminés
- Code committé



📌 À RETENIR

CONTEXT API

- **createContext()** : Créer un contexte
- **Provider** : Fournir les données aux composants enfants
- **useContext()** : Consommer le contexte dans un composant
- **Hook personnalisé** : Encapsuler useContext pour faciliter l'utilisation

QUAND UTILISER CONTEXT ?

✅ Oui :

- État global (auth, theme, langue)
- Données partagées par beaucoup de composants
- Configuration de l'application

❌ Non :

- État local à un composant
- Données qui changent très fréquemment (performance)
- État de formulaire simple

PATTERNS IMPORTANTS

- Un Provider par contexte
- Hook personnalisé pour chaque contexte
- Validation que le contexte est utilisé dans un Provider
- Optimisation avec useMemo si nécessaire

PERFORMANCE

// ⚠ Attention : Re-render à chaque changement

```
<Context.Provider value={{ state, setState }}>
```

// ✅ Mieux : Mémoiser la valeur

```
const value = useMemo(() => ({ state, setState }), [state]);
```

```
<Context.Provider value={value}>
```